

STANDARD OPERATING PROCEDURE

PRD → Production-Quality Mobile App in 2–3 Days

Version: 1.0

Primary development environment: Cursor

Secondary engineering agent: Claude Code

Source of truth: GitHub

Default mobile stack: React Native + Expo + TypeScript

Default backend: Supabase

Default subscriptions: RevenueCat

Default analytics: PostHog

Default monitoring: Sentry

Default E2E testing: Maestro

Default distribution: Expo EAS

1. OBJECTIVE

The purpose of this SOP is to convert an approved Product Requirements Document into a high-quality, fully functioning iOS and Android application as quickly and reliably as possible.

For suitable applications, the target is:

Day 1: Architecture + foundation + primary product loop

Day 2: Feature completion + monetization + integrations + polish

Day 3: QA + security + PRD audit + production builds + distribution

The target outcome is not merely:

“The app runs.”

The target outcome is:

A production-quality release candidate that has been audited against the original PRD, tested on physical devices, instrumented, secured, built for iOS/Android and made available for founder testing or store submission.

2. NON-NEGOTIABLE PRINCIPLE

AI agents do not make product decisions on our behalf.

The hierarchy of authority is:

1. **Approved PRD**
2. **Approved design/brand requirements**
3. **Engineering Contract**
4. **Architecture documentation**
5. **Existing codebase conventions**
6. **Developer judgment**
7. **AI recommendation**

If Cursor or Claude suggests something conflicting with the PRD, the PRD wins unless the developer formally records why the requirement is technically impossible, unsafe or contradictory.

Never silently modify requirements because implementation would be easier.

3. RESPONSIBILITY SPLIT

Cursor = Primary Builder

Cursor is responsible for approximately **80–90% of implementation work**.

Use Cursor for:

- Creating components
- Building screens
- Implementing features
- Database integration
- API integration
- Navigation
- State management
- Forms and validation
- Analytics implementation
- Tests
- Debugging
- Refactoring

- UI polishing
- Terminal commands
- Running tests
- Fixing ordinary build errors

Cursor is the developer's main IDE.

Claude Code = Senior Engineer / Red Team

Claude Code should not duplicate everything Cursor is doing.

Use Claude primarily for:

- Initial architecture review
- Challenging implementation plans
- Difficult bugs
- Complex business logic
- Security review
- Database/RLS review
- Race-condition analysis
- Subscription/payment review
- Code-quality audits
- PRD completeness audits
- Final adversarial QA
- Difficult refactoring
- Investigating problems Cursor repeatedly fails to solve

Claude should normally **review or solve difficult problems**, rather than continuously rewriting Cursor's work.

This reduces:

- Token usage
 - Conflicting changes
 - Duplicate work
 - Architectural drift
-

4. SOURCE OF TRUTH

GitHub is the authoritative codebase.

Never allow the only current version of an application to exist inside:

- Cursor chat
- Claude session
- Replit
- AI Studio
- Local uncommitted files

Every meaningful completed vertical slice must be committed.

Recommended commit pattern:

feat: complete onboarding flow

feat: implement estimate creation

feat: add subscription entitlement handling

fix: prevent duplicate estimate submission

test: add checkout e2e coverage

chore: prepare release candidate

`main` must remain releasable.

For the initial rapid build create:

build/v1

Complete development there and merge to `main` after QA.

5. STANDARD APP STACK

Unless the PRD specifically requires otherwise, use:

Mobile

React Native

Expo

Expo Router

TypeScript strict mode

Backend

Supabase:

- Postgres
- Authentication
- Storage
- Edge Functions
- Row Level Security
- Realtime only when necessary

Subscription monetization

RevenueCat

Product analytics

PostHog

Error/crash monitoring

Sentry

Push notifications

Expo Notifications

End-to-end testing

Maestro

Source control

GitHub

Builds / distribution

Expo EAS

Avoid introducing alternate technology merely because an AI agent prefers it.

6. APP FACTORY REQUIREMENT

Never create an ordinary app from an empty repository unless there is a compelling architectural reason.

Maintain a company starter repository:

mobile-app-starter

This repository should already contain reusable production infrastructure.

Examples:

- Authentication
- Navigation
- Protected routes
- Design system
- Error boundaries
- Analytics wrapper
- Sentry
- Supabase client
- RevenueCat wrapper
- Push notification infrastructure
- Environment management
- Secure storage
- Deep-link infrastructure
- Loading components
- Error states
- Empty states
- Form components
- Profile
- Settings
- Subscription management
- Account deletion
- Terms
- Privacy
- Support
- EAS configuration
- Linting
- Type checking
- Unit testing
- Maestro
- CI

A new app should largely be:

Starter architecture + new product-specific functionality + new branding.

This is one of the main mechanisms that makes the 2–3 day target realistic.

7. STANDARD REPOSITORY STRUCTURE

Use approximately:

```
/
├── app/
│   ├── (auth)/
│   ├── (onboarding)/
│   ├── (tabs)/
│   └── _layout.tsx
├── src/
│   ├── components/
│   │   ├── ui/
│   │   └── shared/
│   ├── features/
│   ├── hooks/
│   ├── services/
│   ├── store/
│   ├── lib/
│   ├── utils/
│   ├── types/
│   └── constants/
├── supabase/
│   ├── migrations/
│   └── functions/
├── maestro/
├── tests/
└── docs/
```

```
| — PRD.md
| — IMPLEMENTATION_PLAN.md
| — ARCHITECTURE.md
| — SCREEN_MAP.md
| — DATABASE.md
| — ANALYTICS.md
| — REQUIREMENTS_MATRIX.md
| — TEST_PLAN.md
| — DECISIONS.md
| — RELEASE_CHECKLIST.md
|
| — .cursor/
|   | — rules/
|
| — .claude/
|
| — CLAUDE.md
| — ENGINEERING_CONTRACT.md
| — README.md
```

8. ENGINEERING CONTRACT

Every project must contain:

ENGINEERING_CONTRACT.md

Both Cursor and Claude must be instructed to obey it.

Use the following contract:

ENGINEERING CONTRACT

You are working on a production application.

`docs/PRD.md` is the authoritative product requirement.

Never

- Remove a PRD requirement to simplify implementation.
- Claim functionality is complete when it is mocked.
- Hard-code secrets.
- Expose private API keys in the client.
- Disable security mechanisms to make development easier.
- Ignore failing tests.
- suppress TypeScript errors merely to obtain a successful build.
- Introduce **any** unnecessarily.
- Rewrite unrelated code.
- install unnecessary packages.
- duplicate existing components.
- implement authorization exclusively in the UI.
- modify production data without explicit authorization.
- commit secrets.
- report a feature as complete unless it actually works.

Every feature must include, where applicable:

- UI
- business logic
- validation
- persistence
- authentication/authorization
- loading state
- empty state
- error state
- offline/network-failure behaviour
- analytics
- accessibility
- tests

Before modifying code:

1. Read the relevant PRD section.
2. Read the relevant architecture.
3. Inspect existing code.
4. Identify reusable components.
5. Plan the change.
6. Implement the smallest correct solution.

After modifying code:

1. Typecheck.
2. Lint.

3. Run applicable automated tests.
4. Test affected user flow.
5. Inspect the diff.
6. Update requirement status.
7. Report unresolved issues.

Production quality takes precedence over shortcuts.

9. CURSOR CONFIGURATION

Create version-controlled Cursor project rules.

Cursor's project rules should point Agent toward:

ENGINEERING_CONTRACT.md
docs/PRD.md
docs/ARCHITECTURE.md
docs/REQUIREMENTS_MATRIX.md

The permanent Cursor instruction should effectively state:

Before implementing any requested work:

1. Read ENGINEERING_CONTRACT.md.
2. Read the relevant PRD requirement.
3. Inspect existing implementation.
4. Implement only the requested scope.
5. Include all relevant edge states.
6. Add or update tests.
7. Run typecheck, lint and relevant tests.
8. Fix failures caused by your work.
9. Update REQUIREMENTS_MATRIX.md when applicable.
10. Report exactly what was changed and anything that remains unverified.

Never treat visual presence as proof of functional completion.

10. CLAUDE CODE CONFIGURATION

Create:

CLAUDE.md

Claude should receive approximately:

You are the senior technical reviewer for this application.

Always read ENGINEERING_CONTRACT.md.

docs/PRD.md is the authoritative product specification.

Your primary responsibilities are:

- architecture analysis
- complex debugging
- security review
- code review
- PRD completeness review
- adversarial QA

Do not unnecessarily rewrite working code.

When reviewing Cursor-generated implementation, assume it may contain subtle failures.

Verify claims against actual code and tests.

Classify findings:

BLOCKER

HIGH

MEDIUM

LOW

A BLOCKER or HIGH issue prevents production release.

When asked to review rather than implement, do not modify files unless explicitly instructed.

11. CLAUDE PERMISSION POLICY

Never run Claude Code with unrestricted destructive permissions as the normal default.

Claude may automatically perform routine safe operations according to the project's approved permission configuration.

Require explicit developer awareness for actions involving:

- destructive database operations
- production deployment
- secrets
- credentials
- deleting substantial file trees
- rewriting Git history
- production migrations
- App Store configuration
- payment configuration
- infrastructure destruction

AI speed must not come at the expense of basic operational safety.

12. STAGE ZERO — PRD INGESTION

No implementation starts immediately after receiving a PRD.

First save the exact approved document as:

docs/PRD.md

Do not replace it with an AI summary.

The original requirements must remain available throughout development.

13. PRD DECOMPOSITION

Start Cursor in planning/review mode.

Use this prompt:

Read:

- ENGINEERING_CONTRACT.md
- docs/PRD.md

Do not write application code yet.

Act as the lead software architect.

Convert the PRD into an implementation specification.

Create/update:

docs/IMPLEMENTATION_PLAN.md
docs/ARCHITECTURE.md
docs/SCREEN_MAP.md
docs/DATABASE.md
docs/ANALYTICS.md
docs/TEST_PLAN.md
docs/REQUIREMENTS_MATRIX.md

Identify:

1. every screen
2. every user journey
3. every functional requirement
4. every business rule
5. every database entity
6. every API/integration
7. every authentication requirement
8. every authorization requirement
9. every subscription/payment behaviour
10. every notification
11. every analytics event
12. every loading state
13. every empty state
14. every failure state
15. every permission
16. every dependency
17. every ambiguity
18. every release-blocking risk

Do not simplify the PRD.

Do not implement anything yet.

14. REQUIREMENTS MATRIX

This is mandatory.

`REQUIREMENTS_MATRIX.md` should contain:

ID	PRD Requirement	Screen/Flo w	Backend	Analytics	Tests	Status
R-001	User registration	Auth	Supabase Auth	signup_completed	E2E-01	Pending
R-002	Create estimate	Estimate	estimates table	estimate_created	E2E-04	Pending

Statuses:

PENDING
IN PROGRESS
IMPLEMENTED
VERIFIED
BLOCKED

Only VERIFIED counts as complete.

15. ASSUMPTIONS LOG

Do not constantly stop the build for minor ambiguities.

Create:

docs/DECISIONS.md

For non-critical ambiguity, make the most sensible production-quality assumption and record:

Decision:
Reason:
PRD implication:
Reversible: Yes/No

Escalate only ambiguities that materially affect:

- business model
- pricing
- legal requirements
- fundamental UX
- security
- architecture
- irreversible implementation

This prevents unnecessary delays.

16. CLAUDE ARCHITECTURE REVIEW

Before major implementation, Claude performs one read-only review.

Prompt:

Read:

ENGINEERING_CONTRACT.md
docs/PRD.md
docs/IMPLEMENTATION_PLAN.md
docs/ARCHITECTURE.md
docs/DATABASE.md
docs/REQUIREMENTS_MATRIX.md

Do not write code.

You are the senior technical architect reviewing Cursor's implementation plan.

Find:

- missing requirements
- unnecessary complexity
- architectural weaknesses

- security issues
- data-model problems
- subscription design problems
- likely implementation bottlenecks
- requirements Cursor misunderstood
- anything likely to prevent completion within 2–3 days

Return findings classified as:

BLOCKER

HIGH

MEDIUM

LOW

Then recommend the smallest production-quality architecture that fully satisfies the PRD.

Fix BLOCKER/HIGH planning issues before implementation.

Do not spend hours debating LOW issues.

17. 2–3 DAY FEASIBILITY GATE

After decomposition, classify the build:

GREEN

Suitable for 2–3 days.

Typical:

- CRUD applications
- Consumer utility apps
- AI wrappers
- Content apps
- Productivity apps
- Subscription tools
- Standard API applications
- Camera/photo apps using existing APIs
- Standard marketplace-lite workflows

AMBER

Possible but aggressive.

Examples:

- significant offline functionality
- complex realtime behaviour
- advanced media workflows
- many external integrations
- unusually sophisticated animations

RED

Do not pretend it is a genuine 2–3 day production build.

Examples:

- advanced video editor
- real-time multiplayer
- major social network
- medical diagnostic software
- banking infrastructure
- complex trading application
- safety-critical software
- custom ML infrastructure
- large two-sided marketplace
- deep hardware integration

For RED projects, the developer should still build as efficiently as possible but must not sacrifice reliability merely to meet an arbitrary clock.

18. DAY 1 — BUILD THE PRODUCT CORE

The goal of Day 1 is:

A user can install the app and complete the product's primary value-producing journey end-to-end.

Do not spend Day 1 producing twenty disconnected screens.

19. BUILD VERTICAL SLICES

Always implement features vertically.

Bad:

Build all screens

↓

Build backend

↓

Connect everything

↓

Discover architecture is broken

Correct:

Screen

↓

Business logic

↓

Backend

↓

Persistence

↓

Analytics

↓

Error states

↓

Tests

↓

Working journey

Then build the next slice.

Example:

Register

→ Create account

→ Create DB profile

→ Enter app

Then:

Create project

→ Validate

→ Save

→ Retrieve

→ Display

Then:

Reach free limit

→ Paywall

→ Subscribe

→ Unlock entitlement

20. CURSOR FEATURE PROMPT

For every substantial feature, use approximately:

Implement PRD requirement(s):

[R-###]

Read first:

ENGINEERING_CONTRACT.md

docs/PRD.md

docs/ARCHITECTURE.md

docs/DATABASE.md

docs/REQUIREMENTS_MATRIX.md

Inspect the existing code before editing.

Implement the complete vertical slice.

Include where applicable:

- UI
- business logic
- validation
- database/API integration
- authentication/authorization

- loading state
- empty state
- error state
- offline/network-failure behaviour
- analytics
- accessibility
- unit/integration/E2E coverage

Reuse existing components.

Do not change unrelated functionality.

When finished:

1. run typecheck
2. run lint
3. run relevant tests
4. fix failures caused by your work
5. inspect your diff
6. update REQUIREMENTS_MATRIX.md
7. report files changed
8. report tests executed
9. report anything that remains unverified

Do not call the feature complete merely because the UI renders.

This is preferable to:

“Build the whole app.”

21. DEVELOPMENT ORDER

Unless the PRD requires otherwise, use:

Phase A — Foundation

- project configuration
- branding
- design tokens
- navigation

- backend
- environment
- auth

Phase B — Core Loop

Build the single most important reason the user downloads the application.

Phase C — Persistence

Ensure data correctly survives:

- navigation
- backgrounding
- logout/login
- app restart where applicable

Phase D — Monetization

Implement subscription/paywall.

Phase E — Secondary Features

Profile, settings, history, search, notifications, etc.

Phase F — Polish

Visual refinement and edge cases.

22. UI DEVELOPMENT STANDARD

Do not allow every screen to invent its own styling.

Establish design tokens immediately:

colors
spacing
font sizes
font weights
radii
shadows

button heights
input heights
animation durations

Build reusable components.

At minimum:

Button
TextButton
IconButton
Input
TextArea
Card
Modal
BottomSheet
Toast
Alert
Spinner
Skeleton
EmptyState
ErrorState
Avatar
Badge
Tabs
Header
ListItem
Paywall

Do not duplicate essentially identical UI.

23. UI QUALITY STANDARD

The developer must review every screen for:

- consistent spacing
- typography
- touch target size
- safe areas
- keyboard avoidance

- keyboard dismissal
- scrolling
- status bar
- long text
- small screen
- large screen
- dark/light mode if required
- loading feedback
- error feedback
- destructive confirmation
- disabled states
- button feedback
- animations
- haptics where appropriate
- accessibility labels
- dynamic content length

The app must not look like an unpolished AI prototype.

24. DATABASE STANDARD

All database changes should be reproducible.

Use migrations.

Do not rely upon:

“I manually created the table in the dashboard.”

Production tables should include appropriate:

- primary keys
- foreign keys
- constraints
- timestamps
- indexes
- unique constraints
- defaults

Use Row Level Security where appropriate.

Explicitly test:

User A cannot access User B's private data.

UI hiding is not authorization.

25. API SECURITY

For secret external APIs:

Correct:

Mobile App



Authenticated backend/Edge Function



External provider



Backend validation



Mobile App

Wrong:

Mobile App containing secret API key



External provider

Never ship private API secrets inside the client bundle.

26. SUBSCRIPTION STANDARD

RevenueCat should ordinarily control mobile subscription entitlement.

The application should reason in terms of:

premium

pro

not:

monthly_product_9_99

throughout the application.

Implement:

- packages/offerings
- monthly option if required
- annual option if required
- trials if required
- purchase
- purchase cancellation
- purchase errors
- entitlement refresh
- restore purchases
- subscription management
- logged-in identity mapping
- paywall analytics
- entitlement-gated functionality

Test actual purchase behaviour in appropriate Apple/Google sandbox environments.

Do not sign off subscription functionality based only on mocked Expo Go behaviour.

27. ANALYTICS STANDARD

Define events before release.

At minimum:

app_opened
onboarding_started
onboarding_completed
signup_started
signup_completed
login_completed

core_action_started
core_action_completed

core_action_failed

paywall_viewed

plan_selected

purchase_started

purchase_completed

purchase_failed

purchase_cancelled

restore_started

restore_completed

Product-specific events should be added according to the PRD.

Include useful properties such as:

source

screen

plan

result

duration

feature

Do not send sensitive personal information unnecessarily.

28. ERROR MONITORING

Sentry must be functional before release.

Capture:

- crashes
- uncaught exceptions
- failed critical operations
- important API failures
- unexpected application states

Verify monitoring by deliberately generating a safe test error in staging.

Do not merely install the SDK and assume it works.

29. EVERY IMPORTANT UI MUST HAVE STATES

Check each significant feature for:

Loading

Loaded

Empty

Success

Validation error

Server error

Offline/network error

Unauthorized/expired session where applicable

Do not leave blank screens.

30. CLAUDE CODE DURING IMPLEMENTATION

Do not open Claude every twenty minutes.

Escalate when:

1. Cursor has unsuccessfully attempted a bug several times.
2. The architecture seems questionable.
3. A security-sensitive implementation has been completed.
4. Subscription logic is behaving unexpectedly.
5. State synchronization becomes complex.
6. The developer suspects a race condition.
7. A refactor could impact many files.

8. A major integration is unclear.

Example debugging prompt:

Act as the senior debugging engineer.

Do not modify anything initially.

Read ENGINEERING_CONTRACT.md and investigate this problem:

[BUG]

Cursor attempted:

[SUMMARY]

Determine the actual root cause from the code.

Do not merely treat symptoms.

Return:

1. root cause
2. affected files
3. safest fix
4. regression risk
5. tests required

After identifying the cause, implement the fix only if requested.

31. USING PARALLEL AGENTS

Parallelize only independent work.

Good:

Agent A → isolated feature

Agent B → tests

Agent C → read-only security audit

Agent D → analytics review

Bad:

Agent A → navigation

Agent B → navigation

Agent C → global auth

Agent D → global auth

Avoid multiple agents changing the same foundational files.

If Claude worktrees or Cursor cloud agents are used, isolate the branches and review diffs before merging.

Never blindly merge AI-generated branches.

32. DAY 1 EXIT CRITERIA

By end of Day 1:

- app launches
- navigation works
- authentication works if applicable
- database foundation works
- main user journey works end-to-end
- primary data persists
- core analytics exists
- major architecture has stabilized
- developer has run the app on at least one real or representative mobile environment
- meaningful work is committed

Target:

60–75% of meaningful product functionality complete.

If the core user journey still does not work, do not spend time polishing settings screens.

33. DAY 2 — COMPLETE AND POLISH

Day 2 should finish:

- remaining core features
- secondary flows
- subscriptions
- notifications
- settings
- analytics
- error monitoring
- permissions
- deep links
- edge cases
- UI polish

By mid-to-late Day 2, no major architecture should still be under construction.

34. FIRST COMPLETE BUILD

As soon as the product becomes feature-complete:

Run:

TypeScript

Lint

Unit tests

Integration tests

E2E

Then create an actual development/preview build.

Do not wait until the final hour to discover native build failures.

35. PHYSICAL DEVICE TESTING

At minimum test on:

One physical iPhone

and

One physical Android device

where practical before release.

Do not rely exclusively on:

- web previews
 - Expo Go
 - emulator
 - screenshots
 - AI claims
-

36. MANUAL QA SCRIPT

Run:

Installation

Fresh install.

Authentication

- signup
- login
- logout
- incorrect password
- forgot password where applicable
- session persistence

Core Journey

Complete the application's primary value proposition from beginning to end.

Persistence

Close and reopen.

Check state.

Invalid Data

Enter invalid inputs.

Network Failure

Test degraded/no connectivity where practical.

Repeated Actions

Tap important actions rapidly.

Ensure duplicate records/purchases are not generated.

Backgrounding

Begin an important operation.

Background the app.

Return.

Permissions

Allow and deny applicable permissions.

Subscription

- open paywall
- select package
- test purchase
- unlock entitlement
- restart app
- verify entitlement
- restore

Account

- edit profile
- settings
- logout
- deletion if required

37. AUTOMATED E2E TESTS

Maestro should cover the highest-value journeys.

At minimum where applicable:

E2E-01 fresh install/onboarding
E2E-02 signup/login
E2E-03 primary feature
E2E-04 data persistence/history
E2E-05 paywall entry
E2E-06 premium entitlement
E2E-07 settings/logout

Do not attempt to automate every cosmetic scenario in a three-day build.

Automate the journeys where regression would materially damage the product.

38. DAY 3 — FREEZE FEATURES

At the beginning of Day 3:

Feature development stops unless something required by the PRD is genuinely missing.

Day 3 exists for:

- bugs
- tests
- security
- QA
- performance
- polish
- production configuration
- release

Do not introduce speculative nice-to-have functionality.

39. CLAUDE ADVERSARIAL CODE REVIEW

Run Claude Code in review mode.

Prompt:

You are now the senior production-readiness engineer.

DO NOT modify code.

Your objective is to prove that this application is not ready for production.

Read:

ENGINEERING_CONTRACT.md
docs/PRD.md
docs/ARCHITECTURE.md
docs/DATABASE.md
docs/REQUIREMENTS_MATRIX.md
docs/TEST_PLAN.md

Then inspect the actual implementation.

Find:

- missing requirements
- partial implementations
- mocked functionality
- crashes
- broken journeys
- authentication vulnerabilities
- authorization vulnerabilities
- insecure API usage
- exposed secrets
- Supabase RLS weaknesses
- subscription bypasses
- race conditions
- stale state
- duplicate submission risks
- incorrect persistence
- missing validation
- bad error handling
- missing loading states
- offline issues
- navigation traps
- memory/performance problems
- accessibility problems

- platform-specific iOS/Android problems
- insufficient tests
- analytics omissions
- monitoring omissions

Do not trust comments or documentation.

Verify actual code.

Classify every finding:

BLOCKER
HIGH
MEDIUM
LOW

For each finding provide:

- evidence
 - affected file/location
 - reproduction conditions
 - recommended fix
 - test required to prove resolution
-

40. FIX POLICY

Before release:

BLOCKER: Must fix.

HIGH: Must fix.

MEDIUM: Fix unless there is a documented reason to defer.

LOW: May defer.

Do not allow AI to “resolve” findings by removing tests or weakening validation.

41. SECURITY AUDIT

Claude should separately inspect:

Authentication
Authorization
RLS
Secrets
API keys
Edge Functions
File/storage permissions
User-controlled input
Account deletion
Subscription enforcement
Deep links
Sensitive logs

Prompt:

Perform a hostile security review of this application.

Assume the attacker:

- controls their client
- can inspect network requests
- can modify local state
- can call backend endpoints directly
- may attempt to access another user's records
- may attempt to bypass premium restrictions

Identify whether server-side controls prevent these attacks.

Do not assume UI restrictions provide security.

42. PRD COMPLIANCE AUDIT

This is mandatory.

Give Claude the original PRD.

Prompt:

Perform a requirement-by-requirement compliance audit.

Compare docs/PRD.md against the ACTUAL application implementation.

For every requirement classify:

VERIFIED
PARTIALLY IMPLEMENTED
NOT IMPLEMENTED
CANNOT VERIFY

Provide:

- PRD requirement
- classification
- implementation evidence
- relevant files
- test evidence
- missing behaviour if any

Do not mark something VERIFIED merely because:

- a screen exists
- a function exists
- a comment claims support
- a test is mocked

VERIFIED means implementation exists and credible evidence demonstrates that it works.

Update **REQUIREMENTS_MATRIX.md**.

Release requires all material PRD requirements to be:

VERIFIED

43. CURSOR FINAL FIX PASS

Give Cursor Claude's BLOCKER/HIGH findings.

Prompt:

Read the attached production-readiness findings.

Resolve only BLOCKER and HIGH findings first.

For each:

1. reproduce/verify the problem
2. implement root-cause fix
3. add regression test
4. run relevant tests
5. report evidence of resolution

Do not weaken tests or requirements to obtain a pass.

Then have Claude verify fixes rather than trusting Cursor's claims.

44. PERFORMANCE PASS

Check:

- unnecessary rerenders
- enormous images
- unbounded lists
- missing pagination where needed
- excessive network calls
- duplicate requests
- expensive work on render
- blocked UI thread
- unnecessary realtime subscriptions
- memory-heavy objects
- startup time

Do not prematurely optimize ordinary code.

Fix material user-visible performance issues.

45. RELEASE CONFIGURATION

Verify:

App name
Bundle identifier
Android package
Version
Build number
App icon
Splash
Deep links
Production API URLs
Supabase production project
RevenueCat production configuration
Sentry production environment
PostHog production environment
Push credentials
Permissions
Privacy strings
Support URL/email
Privacy Policy
Terms
Account deletion

No staging credentials should accidentally ship to production.

46. ENVIRONMENT STANDARD

Maintain:

development
preview/staging
production

Never use production as the developer's ordinary test environment.

EAS profiles should correspond appropriately.

Example concept:

development
preview
production

Production secrets must not live in Git.

47. PRODUCTION BUILD

Create actual production-equivalent binaries.

Target:

iOS → TestFlight

Android → Google Play Internal Testing

or appropriate internal EAS distribution earlier in QA.

Do not mark the app complete because:

`npx expo start`

works.

Production build success is a release gate.

48. FOUNDER ACCEPTANCE BUILD

Provide an installable release candidate.

The founder should not need:

- Cursor
- source code
- Expo development environment

- developer intervention

to experience the product.

They should install the actual application.

49. FOUNDER ACCEPTANCE TEST

Founder/product owner tests approximately:

Test A

Install → onboard → account.

Test B

Perform primary value-producing action.

Test C

Repeat core action.

Test D

Close/reopen and verify persistence.

Test E

Encounter monetization → subscribe → premium unlock.

Test F

Restore subscription where relevant.

Test G

Test the app from a fresh user perspective without developer guidance.

A failure in the primary product journey prevents release.

50. RELEASE CHECKLIST

Before submission confirm:

- ☐ PRD stored unchanged
 - ☐ requirements matrix completed
 - ☐ all material requirements VERIFIED
 - ☐ zero known BLOCKER defects
 - ☐ zero known HIGH defects
 - ☐ TypeScript passes
 - ☐ lint passes
 - ☐ unit/integration tests pass
 - ☐ critical Maestro tests pass
 - ☐ physical iOS QA completed
 - ☐ physical Android QA completed
 - ☐ subscriptions sandbox-tested
 - ☐ authentication tested
 - ☐ RLS/security checked
 - ☐ secrets checked
 - ☐ production environment checked
 - ☐ PostHog verified
 - ☐ Sentry verified
 - ☐ account deletion works if required
 - ☐ privacy/terms present
 - ☐ production iOS build succeeds
 - ☐ production Android build succeeds
 - ☐ TestFlight/internal build available
 - ☐ founder acceptance test completed
-

51. STANDARD 3-DAY TIMETABLE

DAY 1 — ARCHITECTURE + CORE LOOP

08:00–09:00

PRD ingestion
Requirements matrix
Architecture

Screen map
Database design

09:00–09:30

Claude architecture review.

09:30–10:30

Clone App Factory.
Configure branding/environment/backend.

10:30–13:00

Authentication + onboarding + first vertical slice.

13:00–17:00

Primary product journey.

17:00–19:00

Backend/persistence + core tests + first build.

Day 1 target:

Core product genuinely works.

DAY 2 — COMPLETE PRODUCT

08:00–12:00

Remaining primary features.

12:00–14:00

RevenueCat / monetization.

14:00–16:00

Secondary features + notifications/integrations.

16:00–18:00

Analytics + Sentry + edge states.

18:00–20:00

UI polish + initial E2E + physical-device test.

Day 2 target:

Feature-complete release candidate.

DAY 3 — BREAK IT AND SHIP IT

08:00–10:00

Full automated/manual QA.

10:00–11:30

Claude adversarial audit.

11:30–14:00

Cursor BLOCKER/HIGH fixes.

14:00–15:00

Claude security review.

15:00–16:00

PRD compliance audit.

16:00–17:00

Final regression tests.

17:00–18:00

Production builds.

Final stage

TestFlight
Google Play Internal Testing
Founder acceptance
Store submission

52. 2-DAY FAST TRACK

A 2-day build is acceptable only when:

- starter repository handles most infrastructure
- PRD is clear
- app architecture is conventional
- integrations are straightforward
- design system is already established
- no complex native/hardware functionality exists

In that case:

Day 1: Build.

Day 2 morning: Complete.

Day 2 afternoon/evening: Audit/test/release.

Never achieve a 2-day target by deleting QA.

53. TOKEN/COST DISCIPLINE — CURSOR

Use Cursor as the everyday implementation tool.

Do not continuously:

- reopen entire PRD context unnecessarily
- ask five agents to solve the same problem
- run heavyweight models for trivial edits
- ask Agent to rewrite the entire app
- spawn parallel agents without clear ownership

Give Cursor bounded tasks.

54. TOKEN/COST DISCIPLINE — CLAUDE CODE

Claude is the escalation layer.

Use it for high-value reasoning.

Good use:

- Review this architecture.

- Find this race condition.

- Audit Supabase security.

- Determine why this bug persists.

- Compare implementation against PRD.

- Red-team this release.

Wasteful use:

- Rewrite this button.

- Rename this variable.

- Add margin.

- Create another standard form.

Routine implementation belongs in Cursor.

55. CONTEXT MANAGEMENT

Do not rely on a 12-hour AI conversation retaining perfect context.

Important decisions belong in repository files:

PRD.md
ARCHITECTURE.md
DATABASE.md
DECISIONS.md
REQUIREMENTS_MATRIX.md

The repository remembers.

The chat does not.

56. NEVER ACCEPT THESE AI STATEMENTS WITHOUT VERIFICATION

If an agent says:

“Everything is working.”

Verify.

If it says:

“All requirements are implemented.”

Audit.

If it says:

“Tests pass.”

Run them.

If it says:

“The database is secure.”

Inspect RLS.

If it says:

“Subscriptions work.”

Sandbox-test them.

If it says:

“Production build should succeed.”

Build it.

AI output is evidence to investigate, not proof.

57. BUG ESCALATION PROTOCOL

When Cursor fails twice on essentially the same problem:

Stop prompting it with variations of “try again.”

Do this:

Step 1

Revert unnecessary speculative edits.

Step 2

Document:

Expected:

Actual:

Reproduction:

Relevant logs:

Attempts:

Step 3

Give the problem to Claude Code as a root-cause investigation.

Step 4

Have Claude propose the fix.

Step 5

Either Claude or Cursor implements exactly one coherent fix.

Step 6

Add regression test.

This prevents AI debugging loops that waste hours.

58. DESIGN CHANGE POLICY

During the 2–3 day sprint:

PRD-required UX:

Must implement.

Obvious usability defect:

Fix.

Nice-to-have design idea:

Backlog.

Do not allow AI or developer enthusiasm to cause scope creep.

59. DEFINITION OF FEATURE COMPLETE

A feature is not complete because:

- screen exists
- button exists
- API call exists
- database table exists

Feature complete means:

UI
+
Logic
+
Persistence
+
Security
+
Error handling
+
Analytics
+
Testing
=
Complete feature

60. DEFINITION OF APP COMPLETE

The app is complete only when:

The approved PRD has been implemented and audited requirement-by-requirement; the primary user journeys function end-to-end on production-equivalent iOS and Android builds; backend authorization and data access have been reviewed; subscription functionality has been tested where applicable; analytics and crash monitoring have been verified; critical automated tests pass; no BLOCKER or HIGH defects remain; and a release candidate has been distributed to TestFlight/Google Play Internal Testing or submitted for store review.

61. GOLDEN RULE

The company is not trying to produce the **largest quantity of code in 72 hours**.

The company is trying to produce:

the smallest, cleanest, fully functioning implementation of the approved PRD that we would be comfortable spending serious acquisition capital to put in front of real users.

Cursor accelerates implementation.

Claude challenges implementation.

GitHub preserves truth.

Tests provide evidence.

The PRD defines completion.

The developer remains accountable for the product that ships.